

Lab Assignment

The Rectangle Class — Rule of Three & Deep Copy

Topics: Static Members • Friend Functions • Copy Constructor • operator= • Destructor • Deep Copy

1. Background

1.1 The Problem with Pointer Data Members

Most C++ classes store their data in simple value-type members such as `int`, `double`, or `string`. When such a class is copied, C++ generates a default copy constructor and assignment operator that perform a **shallow copy** — they duplicate the value of every member directly. For value types this is perfectly safe.

The situation changes when a class contains a **pointer** to heap-allocated memory. A shallow copy duplicates the pointer *address* rather than the data it points to. Both the original and the copy then share the same heap block, which leads to two serious defects:

- ▶ **Double-free error** — when both objects are destroyed their destructors both call `delete` on the same address, corrupting the heap.
- ▶ **Dangling pointer** — modifying the data through one object silently changes the other, because they share memory.

The `Rectangle` class in this lab deliberately introduces a pointer member `string* pcolor` to force exactly this scenario. Your task is to implement the three special member functions that together constitute the **Rule of Three** and guarantee safe copying.

1.2 The Rule of Three

Whenever a class manually manages a resource (heap memory, a file handle, a network socket), it almost certainly needs all three of the following:

Special Function	Purpose
Destructor <code>~Rectangle()</code>	Release heap memory when the object goes out of scope or is deleted. Without this, every <code>Rectangle</code> leaks its <code>pcolor</code> string.
Copy Constructor <code>Rectangle(const Rectangle&)</code>	Allocate fresh heap memory for the new object and copy the data from the source — a deep copy. Without this, two <code>Rectangle</code> objects share one <code>pcolor</code> block.
Copy-Assignment Operator <code>operator=</code>	Free the current heap block, allocate a new one, and copy the data from the right-hand side. Must include a self-assignment guard.

1.3 Static Members and Friend Functions

Static data members belong to the class, not to any particular object. In this lab, `int Rectangle::counter` tracks how many `Rectangle` objects currently exist. It is incremented in every constructor and decremented in the destructor, giving a live count at any point in the program.

Static member functions can be called without an object instance: `Rectangle::getCount()`. They may only access static data members, since there is no `this` pointer.

Friend functions are non-member functions granted access to a class's private members. In this lab, `showBox(Rectangle& r)` and `operator<<` are declared friends so they can read `r.length` and `r.width` directly without going through public accessors. A function must be declared a friend *inside the class* to receive this privilege — being a friend of one class does not confer any special access to other classes.

1.4 Deep Copy vs. Shallow Copy

The diagram below contrasts what happens to `pcolor` under each copy strategy:

Shallow copy (DANGEROUS):

```
graph LR
    r1_pcolor[r1.pcolor] --> pink_block["Pink"]
    r2_pcolor[r2.pcolor] --> pink_block
```

Both point to the same block.
Deleting `r1` leaves `r2` with a dangling pointer!

Deep copy (CORRECT):

```
graph LR
    r1_pcolor[r1.pcolor] --> pink_block1["Pink"]
    r2_pcolor[r2.pcolor] --> pink_block2["Pink"]
    pink_block1 -- separate block --> pink_block2
```

Each owns its own block.
Deleting `r1` has no effect on `r2`.

In the copy constructor the sequence is: (1) allocate a new `string` on the heap, (2) copy the characters from the source. In the assignment operator the sequence is: (1) self-assignment guard, (2) `delete` the current block, (3) allocate a new block, (4) copy the characters.

2. Class Specification

Implement the `Rectangle` class in a header file named `Rectangle.h`. The class must match the specification below exactly, since the provided `main.cpp` relies on this interface.

2.1 Private Data Members

```
double length;           // always non-negative (use abs())
double width;            // always non-negative
string* pcolor = nullptr; // heap-allocated color name
static int counter;      // counts live Rectangle objects
```

2.2 Public Interface

Mutators

```
void setLength(double lengthValue); // store abs(lengthValue)
```

```
void setWidth (double widthValue);    // store abs(widthValue)
void setColor (string colorValue);    // assign via *pcolor
```

Accessors

```
double getLength() const;
double getWidth() const;
string getColor() const;
```

Constructors, Destructor, and Assignment

```
// Default/parameterized constructor
Rectangle(double lengthValue = 0,
          double widthValue = 0,
          string colorValue = "White");

// Copy constructor (deep copy)
Rectangle(const Rectangle& other);

// Copy-assignment operator (deep copy, self-assignment guard)
Rectangle& operator=(const Rectangle& other);

// Destructor (release heap memory, decrement counter)
~Rectangle();
```

Other Methods

```
string toString() const;    // see format below
static int getCount();      // returns counter
```

Friend Declarations

```
friend void    showBox(Rectangle& r);
friend ostream& operator<<(ostream& sout, const Rectangle& r);
```

2.3 toString() Format

The `toString()` method must produce output in the following format (address values will differ on each run):

```
123ABCDEF Rectangle [Length: 11, Width: 11,
pcolor value: Pink, pcolor addr: 0x55a3b2c0]
```

Use `stringstream` and `this` (the object's own address) to build the string. Include both the dereferenced `pcolor` value and the raw pointer address so that students can verify independent heap blocks in `experiment03`.

2.4 Static Counter Initialization

Place the following line *outside* the class body, at file scope in `Rectangle.h`:

```
int Rectangle::counter = 0;
```

2.5 friend function: showBox()

Implement `showBox(Rectangle& r)` in `main.cpp` (outside the class). It has direct access to the private members `r.length` and `r.width` because it is declared a friend inside the class. An equivalent non-friend function would require the public accessors and would fail to compile if it attempted direct member access.

3. Required Experiments

Your `main.cpp` must contain the three experiment functions below. Each must be self-contained and callable independently from `main()`. Use the exact function signatures shown.

Experiment 01 — Static Counter and Friend Function

Demonstrate the static `counter` member and the `showBox` friend function.

Tasks to verify:

- Call `Rectangle::getCount()` before creating any object — it must return 0.
- Create `r1(10, 10)`. Verify counter increments to 1.
- Declare an array `Rectangle a[3]{ r1 }`. Verify counter becomes 4 (1 original + 3 copies).
- Print all elements of the array using `toString()`.
- Create a heap `Rectangle` via `new` and print its address and `toString()`.
- Call `showBox(r1)` and confirm it prints length and width directly from private members.

```
void experiment01() {
    cout << "Total rectangles: " << Rectangle::getCount() << endl;
    Rectangle r1(10, 10);
    cout << "Total rectangles: " << Rectangle::getCount() << endl;
    Rectangle a[3]{ r1 };
    cout << "Total rectangles: " << Rectangle::getCount() << endl;
    for (Rectangle r : a)
        cout << r.toString() << endl;
    Rectangle* p1 = new Rectangle(20, 20);
    cout << "p1    " << p1 << endl;
    cout << "*p1  " << p1->toString() << endl;
    showBox(r1);
}
```

Experiment 02 — Objects and Pointers to Objects

Explore the difference between a Rectangle object and a pointer to a Rectangle. Observe how changes made through a pointer affect the original object.

Tasks to verify:

- ▶ Create `r1(11, 11)`. Print its `toString()`.
- ▶ Declare `Rectangle* p1 = nullptr` and print `p1` (should be 0 / null).
- ▶ Assign `p1 = &r1`. Both `r1.getLength()` and `p1->getLength()` must agree.
- ▶ Call `p1->setWidth(77)` and confirm that `r1.toString()` also reflects the change (same object).
- ▶ Reassign `p1 = new Rectangle(55, 66)` (`p1` now points to a different object). Confirm `r1` is unaffected.
- ▶ Call `delete p1` and observe the destructor message.

```
void experiment02() {
    Rectangle r1(11, 11);
    cout << "r1  " << r1.toString() << endl;
    Rectangle* p1 = nullptr;
    cout << "p1  " << p1 << endl;
    p1 = &r1;
    cout << "p1  " << p1 << endl;
    cout << "r1 length: " << r1.getLength() << endl;
    cout << "r1 length: " << p1->getLength() << endl;
    p1->setWidth(77);
    cout << "r1  " << r1.toString() << endl;
    cout << "*p1 " << p1->toString() << endl;
    r1.setWidth(88);
    cout << "r1  " << r1.toString() << endl;
    p1->setWidth(99);
    cout << "r1  " << r1.toString() << endl;
    p1 = new Rectangle(55, 66);
    cout << "r1  " << r1.toString() << endl;
    cout << "*p1 " << p1->toString() << endl;
    delete p1;
}
```

Experiment 03 — Deep Copy (Rule of Three)

Verify that your copy constructor and copy-assignment operator perform a genuine deep copy by confirming that each Rectangle object owns an independent `pcolor` heap block.

Tasks to verify:

- ▶ Create `r1(11, 11, "Pink")` and print its `toString()`. Note the `pcolor` address.
- ▶ Copy-construct `r2(r1)`. Print `r2.toString()`. Confirm `pcolor` addresses differ.
- ▶ Copy-initialize `r3 = r1` (also invokes copy constructor). Confirm another independent `pcolor` block.
- ▶ Create `r4(2, 4, "Navy Blue")`. Print it, then assign `r4 = r1`.
- ▶ Print `r1` and `r4` side by side. Verify `pcolor` values match but `pcolor` addresses differ.
- ▶ Use `operator<<` on `r4` to confirm the friend stream operator works.

- Observe all four destructor messages as the function returns.

```
void experiment03() {
    Rectangle r1(11, 11, "Pink");
    cout << "r1 " << r1.toString() << endl;

    Rectangle r2(r1);          // copy constructor
    cout << "r2 " << r2.toString() << endl;

    Rectangle r3 = r1;         // copy constructor
    cout << "r3 " << r3.toString() << endl;

    Rectangle r4(2, 4, "Navy Blue");
    cout << "r4 " << r4.toString() << endl;

    r4 = r1;                   // copy-assignment operator
    cout << "r1 " << r1.toString() << endl;
    cout << "r4 " << r4.toString() << endl;
    cout << "r4 " << r4 << endl;    // operator<<
}
```

4. Implementation Notes

- Default parameter values: declare them only in the header, never in the definition.
- `abs()`: use it in `setLength()` and `setWidth()` so negative inputs are silently corrected.
- `pcolor` initialization: always initialize `pcolor = nullptr` in the class. The constructor allocates with `new string(colorValue)`. Never dereference `pcolor` before it is set.
- `setColor()` caution: `setColor()` dereferences `*pcolor`. Only call it after `pcolor` points to a valid heap block. In the copy constructor, call `new string()` first, then `setColor()`.
- Self-assignment guard: in `operator=`, check `(this != &other)` before deleting `pcolor`. Without it, `r = r` deletes the data and then tries to copy from it.
- counter increments: increment counter in every constructor (default and copy). Decrement only in the destructor.
- Static initialization: place `int Rectangle::counter = 0` at file scope in `Rectangle.h`, outside the class body.
- `toString()` address: use this (the implicit pointer) at the start of the string to display the object own address.

Common Mistakes to Avoid

- Calling `setColor()` before allocating `pcolor` causes an immediate crash (null dereference).
- Forgetting the self-assignment guard in `operator=` causes double-free on `r = r`.
- Omitting `counter++` in the copy constructor causes `getCount()` to under-count.
- Using a shallow copy: if `pcolor` addresses match after copying, both destructors free the same block and corrupt the heap.
- Forgetting `delete pcolor` in the destructor leaks memory on every `Rectangle` construction.

5. Deliverables

- Rectangle.h — complete class definition and implementation
- main.cpp — showBox() friend function and all three experiment functions, callable from main()

Reflection Questions

Answer in 2–3 sentences each:

- 1. What is the difference between a shallow copy and a deep copy? Give a concrete example using the Rectangle class.
- 2. Why must the copy-assignment operator check for self-assignment before deleting pcolor? What would happen without this guard?
- 3. In experiment02, when p1 = &r1 and you call p1->setWidth(77), r1 also changes. Explain why. What happens to r1 when p1 is later reassigned to a new Rectangle?
- 4. Why does operator= return Rectangle& instead of Rectangle? What does return *this mean?
- 5. In experiment01, an array of 3 Rectangles is initialized with r1. How many times is the copy constructor called, and why? How does this affect the counter?

Key Test for Deep Copy

In experiment03, print the pcolor address of r1, r2, r3, and r4 after all assignments. Each address must be different. If any two share the same pcolor address, your copy is shallow and both destructors will corrupt the heap when those objects go out of scope.

Future Work.

Define an RGBColor class to represent colors using RGB values. Then, create a new class called RectangleRGB (similar to Rectangle), which, rather than storing a pointer to a color description string, contains its own RGBColor instance (demonstrating composition).